

Assignment 8

Selenium Frameworks, TestNG, Maven

Q1. Explain Data-Driven Testing in Selenium. How can Apache POI be used to read and write data in Excel? Give an example.

Answer:

Data-Driven Testing is a testing technique where test data is stored outside the test script (Excel, CSV, Database, etc.). The same Selenium test runs multiple times using different input values.

Apache POI is a Java library used to read and write Excel (.xlsx) files.

Uses of Apache POI

- Read usernames and passwords from Excel.
- Write test results (Pass/Fail) back to Excel.
- Avoid hardcoding test data.

Example Scenario

Suppose an Excel file contains:

Username	Password
admin	admin123
user1	pass123
user2	test456

Q2. Selenium WebDriver + POM (Page Object Model)

LoginPage.java

```
package pages;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
public class LoginPage {
```

```
    WebDriver driver;
```

```
    By username = By.id("username");
```

```
    By password = By.id("password");
```

```
    By loginBtn = By.id("login");
```

```
    public LoginPage(WebDriver driver) {
```

```
        this.driver = driver;
```

```
    }
```

```
    public void enterUsername(String user) {
```

```
        driver.findElement(username).sendKeys(user);
```

```
    }
```

```
    public void enterPassword(String pass) {
```

```
        driver.findElement(password).sendKeys(pass);  
    }
```

```
public void clickLogin() {  
    driver.findElement(loginBtn).click();  
}
```

```
public void login(String user, String pass) {  
    enterUsername(user);  
    enterPassword(pass);  
    clickLogin();  
}  
}
```

LoginTest.java

```
package tests;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import pages.LoginPage;
```

```
public class LoginTest {
```

```
public static void main(String[] args) {  
  
    WebDriver driver = new ChromeDriver();  
  
    driver.get("https://example.com/login");  
  
    LoginPage lp = new LoginPage(driver);  
  
    lp.login("admin", "admin123");  
  
    driver.quit();  
}  
}
```

Q3. What is TestNG.xml? Write a sample configuration to run 2 test classes in parallel.

Answer

testng.xml is a configuration file used by TestNG to control test execution. It specifies test classes, packages, groups, parameters, and enables parallel execution.

Sample testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
```

```
<suite name="Suite" parallel="classes" thread-count="2">
```

```
<test name="Regression">
```

```
<classes>
```

```
<class name="tests.LoginTest"/>
```

```
<class name="tests.HomeTest"/>
```

```
</classes>
```

```
</test>
```

```
</suite>
```

Explanation:

- `parallel="classes"` → Runs test classes simultaneously.
- `thread-count="2"` → Uses two threads.

Q4. Explain the attributes of `@Test` annotation.

1. priority

Defines the execution order.

```
@Test(priority = 1)
```

```
public void login() {
```

```
    System.out.println("Login");
```

```
}
```

```
@Test(priority = 2)

public void logout() {

    System.out.println("Logout");

}
```

2. dependsOnMethods

Makes one test depend on another.

```
@Test

public void login() {

    System.out.println("Login");

}
```

```
@Test(dependsOnMethods = "login")

public void payment() {

    System.out.println("Payment");

}
```

Payment runs only if Login passes.

3. enabled

Used to enable or disable a test.

```
@Test(enabled = false)

public void register() {

    System.out.println("Register");

}
```

This test will not execute.

Q5. Create a Maven Project for Selenium.

Sample pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.test</groupId>
```

```
<artifactId>SeleniumProject</artifactId>
```

```
<version>1.0</version>
```

```
<dependencies>
```

```
<!-- Selenium -->
```

```
<dependency>
```

```
<groupId>org.seleniumhq.selenium</groupId>
```

```
<artifactId>selenium-java</artifactId>
```

```
<version>4.33.0</version>
```

```
</dependency>
```

```
<!-- TestNG -->
```

```
<dependency>
```

```
  <groupId>org.testng</groupId>
```

```
  <artifactId>testng</artifactId>
```

```
  <version>7.11.0</version>
```

```
  <scope>test</scope>
```

```
</dependency>
```

```
</dependencies>
```

```
</project>
```

Steps to Execute Test Cases Using Maven

1. Create a Maven Project in Eclipse.
2. Add Selenium and TestNG dependencies in pom.xml.
3. Save the file and wait for Maven to download the libraries.
4. Create your Selenium TestNG test classes.
5. Right-click the project → Run As → Maven Test, or run the command:
6. Maven executes the TestNG test cases and displays the results in the Console.